

Datenmanagement in R

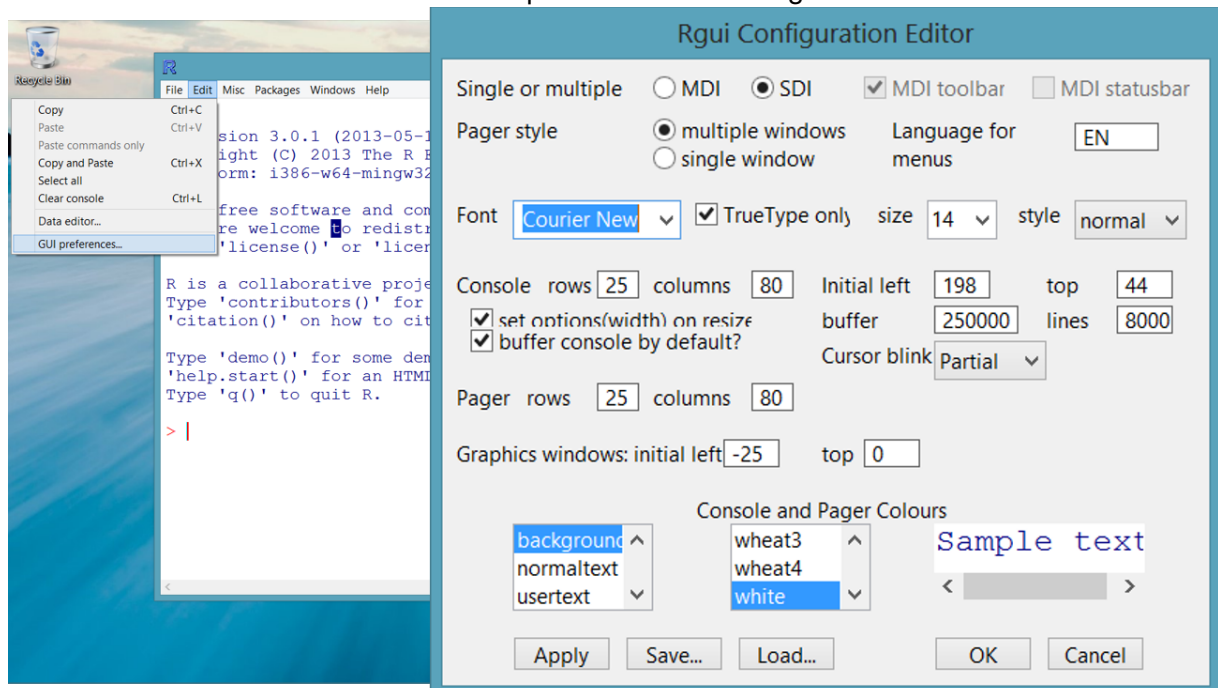
Studienbrief

1 Einführung und Datenmanagement in R

Dieses Kapitel behandelt grundlegende Datenstrukturen in **R**: Wie kann man Variablen anlegen und verändern, wie kann man komplexe Datenstrukturen aufbauen und wie kann man Daten einlesen und exportieren?

1.1 Einführung in R

R ist das aktuell mächtigste Statistik-System. Es besteht aus einem Basissystem und optionalen Erweiterungspaketen ("Packages") - derzeit weit über 8.000! Das System läuft auf allen gängigen Plattformen und ist mehrsprachenfähig. Zentraler Bezugsort ist <http://cran.r-project.org>, eine Open-Source-Plattform. **R** läuft "stand-alone", aber es gibt eine Vielzahl an Editoren, die eine **R**-Integration bieten, z.B. Eclipse (<http://www.walware.de/goto/statet>) oder das speziell entwickelte R-Studio (<http://www.rstudio.com>). Im folgenden ein Screenshot des Konfigurationsfensters des Standard-**R**-GUIs mit empfohlenen Einstellungen.



1.1.1 Ausdrücke

R ist ein interaktiver Interpreter der Sprache **S**, und daher kommandozeilenorientiert. Kommandos sind meist Funktionsaufrufe:

```
cat("Hello, World!")

## Hello, World!
```

(in diesem Fall der Funktion `cat()`), oder Ausdrücke:

```
1 + 1

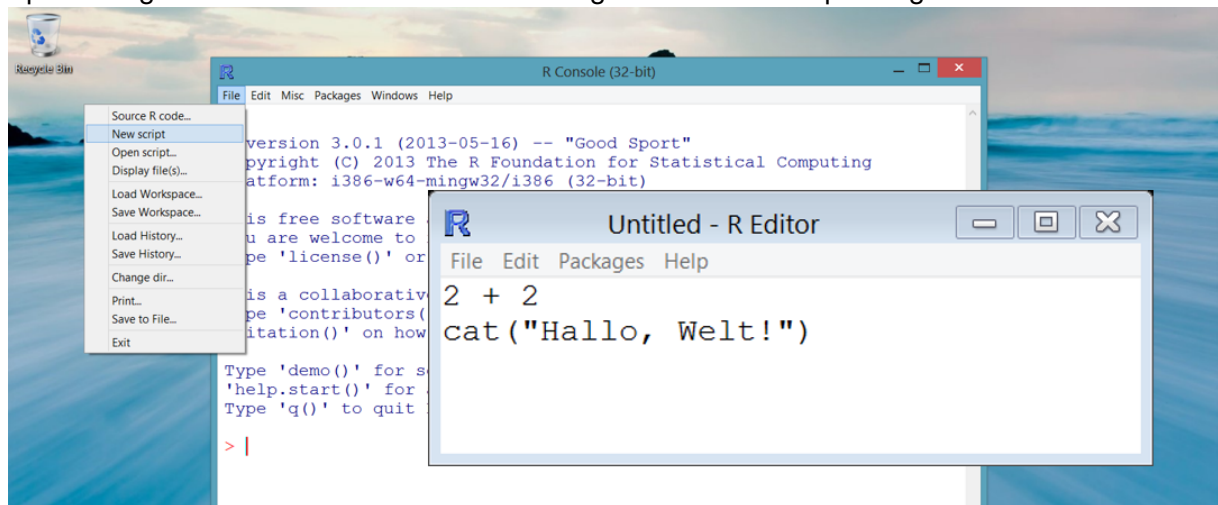
## [1] 2
```

Hilfe zu einer Funktion erhält man mit `?`:

```
?cat
```

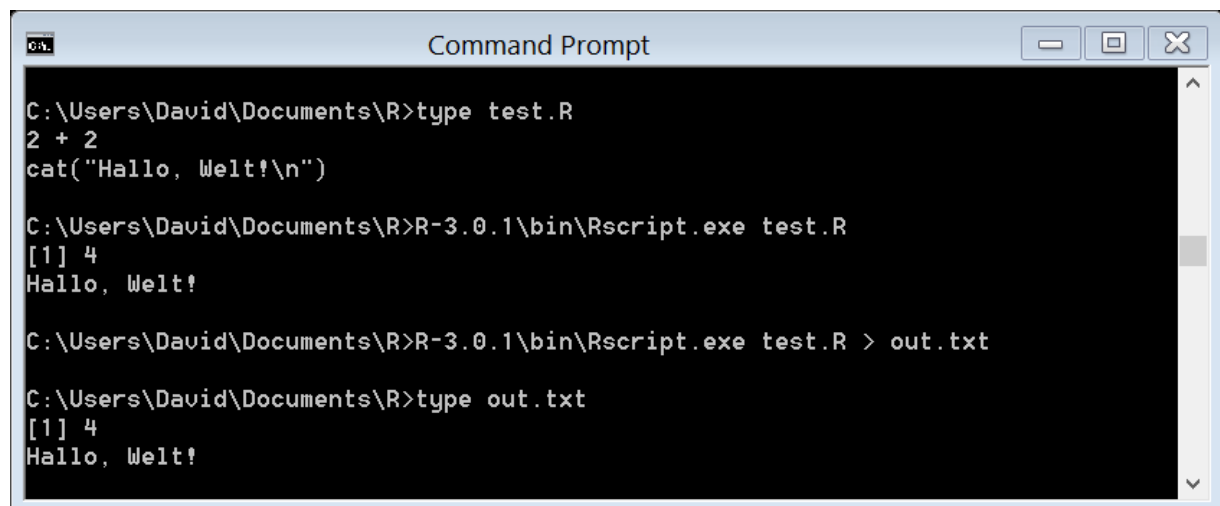
Jede Hilfeseite enthält eine Beschreibung der Parameter und des Rückgabewerts, sowie am Ende Beispiele.

Kommandos können zu einem Skript zusammengefasst werden. Im Editor können diese gespeichert/geladen und markierte Teile mit `Strg + R` an den Interpreter geschickt werden.



R-Skripts können mittels des Programms `Rscript.exe`¹ auch außerhalb des GUIs aufgerufen werden. Der Output kann in eine Textdatei umgeleitet werden. Für Grafiken werden eigene Bilddateien erstellt.

¹Unter Linux mittels `R -f`

A screenshot of a Windows Command Prompt window titled "Command Prompt". The window shows the execution of an R script named "test.R". The first command is "type test.R", which displays the contents of the script: "2 + 2" and "cat('Hallo, Welt!\n')". The second command is "R-3.0.1\bin\Rscript.exe test.R", which executes the script, outputting "[1] 4" and "Hallo, Welt!". The third command is "R-3.0.1\bin\Rscript.exe test.R > out.txt", which runs the script and saves the output to a file named "out.txt". The final command is "type out.txt", which displays the contents of the file: "[1] 4" and "Hallo, Welt!".

```
ca. Command Prompt
C:\Users\David\Documents\R>type test.R
2 + 2
cat("Hallo, Welt!\n")

C:\Users\David\Documents\R>R-3.0.1\bin\Rscript.exe test.R
[1] 4
Hallo, Welt!

C:\Users\David\Documents\R>R-3.0.1\bin\Rscript.exe test.R > out.txt

C:\Users\David\Documents\R>type out.txt
[1] 4
Hallo, Welt!
```

1.1.2 Variablen

Variablen werden durch Zuweisung definiert: Als Zuweisungsoperator kann = oder <- verwendet werden (<- stellt einen Pfeil (←) dar: "x gets 2"). Alle Elemente in **R** (Funktionen, Variablen, Zahlen ...) sind Objekte. Wird ein Objektname eingegeben, wird das Objekt geeignet dargestellt:

```
## Eine ganze Zahl
x <- 2
x = 2
x

## [1] 2
```

(# kennzeichnet Kommentare.)

```
## Ein String
x = "Hugo"
x

## [1] "Hugo"
```

```
## Ausgabe einer Funktion
cat

## function (... , file = "", sep = " ", fill = FALSE, labels = NULL,
##      append = FALSE)
## {
```

```
##      if (is.character(file))
##          if (file == "")
##              file <- stdout()
##          else if (startsWith(file, "|")) {
##              file <- pipe(substring(file, 2L), "w")
##              on.exit(close(file))
##          }
##          else {
##              file <- file(file, ifelse(append, "a", "w"))
##              on.exit(close(file))
##          }
##      .Internal(cat(list(...), file, sep, fill, labels, append))
## }
## <bytecode: 0x5636e13c35a0>
## <environment: namespace:base>
```

Die Ausgabe einer Funktion resultiert in der Darstellung ihres Quellcodes. **R**-Objekte werden im Hauptspeicher im *Workspace* gespeichert. Die aktuell existierenden Objekte können mit `ls()` angezeigt und mit `rm()` gelöscht werden:

```
ls()

## [1] "x"

rm(x, y)

## Warning in rm(x, y): Objekt 'y' nicht gefunden

x

## Error in eval(expr, envir, enclos): Objekt 'x' nicht gefunden
```

(Alle Objekte löscht man mit `rm(list = ls())`). Gespeichert und geladen können entweder einzelne Objekte:

```
a = b = 1
save(a, b, file = "myvars.rda")
rm(a, b)
load("myvars.rda")
a

## [1] 1

b
```

```
## [1] 1
```

oder der ganze Workspace:

```
save.image() # Speichert alles in das File .RData
rm(a, b)
load(".RData")
```

Beim Beenden von **R** wird der Benutzer immer zum Speichern des Workspaces aufgefordert, beim Starten von **R** wird er automatisch geladen.

1.1.3 Vektoren

R arbeitet grundsätzlich *vektoriert*. Ein Vektor ist einfach eine Folge von Elementen gleichen Typs - etwa wie ein Array in Java. Vektoren werden in **R** mit der Funktion `c()` erzeugt:

```
x = c(1, 2, 3)
x

## [1] 1 2 3

y = c(99, x, 99)
y

## [1] 99 1 2 3 99
```

Eine bequeme Möglichkeit, Sequenzen ganzer Zahlen zu erzeugen, ist der `:`-Operator:

```
z = 1 : 50
z

## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
## [21] 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40
## [41] 41 42 43 44 45 46 47 48 49 50

z = 5 : 1
z

## [1] 5 4 3 2 1
```

(Bemerkung: Bei der Ausgabe eines Vektors zeigt die Zahl in eckigen Klammern am Anfang jeder Zeile den Index des ersten Elements in der jeweiligen Zeile an.)

Auf einzelne Elemente von Vektoren kann mit dem `[]`-Operator zugegriffen werden:

```
y[2]

## [1] 1
```

Es können auch mehrere Elemente auf einmal extrahiert werden:

```
y[2 : 4] ## extrahiert die Elemente 2 bis 4

## [1] 1 2 3

ind = c(1, 3, 5)
y[ind] ## Extrahiert das 1., 3. und 5. Element

## [1] 99 2 99
```

Einzelne Elemente können mittels Indexierung verändert werden (auch mit Multiindex):

```
y[1] = 99
y

## [1] 99 1 2 3 99

y[c(1 : 3, 5)] = c(100, 200, 300, 400)
y

## [1] 100 200 300 3 400
```

Negative Index-Elemente entfernen das entsprechende Element. Damit dies dauerhaft geschieht, muss jedoch der entsprechende Vektor neu definiert werden:

```
x = c(1, 2, 3)
x[-2] ## liefert temporaer einen Vektor ohne 2. Element zurueck

## [1] 1 3

x ## Original unveraendert!

## [1] 1 2 3

x = x[-2] ## Dauerhaft loeschen
x

## [1] 1 3
```

Elemente können auch benannt und mit diesen Labels indexiert werden ("assoziatives Array" oder Hash):

```

alter = c(Susi = 10, Hugo = 20)
alter

## Susi Hugo
##    10    20

alter["Susi"]

## Susi
##    10

```

Das interessante an der Vektorisierung in **R** ist, dass die meisten Funktionen auch vektorisiert arbeiten. Damit benötigt man selten Schleifen, um eine Operation auf die Elemente eines Vektors anzuwenden:

```

x = 1:5
1 + x

## [1] 2 3 4 5 6

2 * x

## [1] 2 4 6 8 10

log(x)

## [1] 0.0000000 0.6931472 1.0986123 1.3862944 1.6094379

```

Dies gilt auch für selbstdefinierte Funktionen, sofern diese ihrerseits vektorisierte Funktionen aufrufen:

```

quadrat = function(x) x * x
quadrat(x)

## [1] 1 4 9 16 25

```

(Näheres zu Funktionen kann in Abschnitt 1.3.1 nachgelesen werden). Abschließend ein paar nützliche Hilfsfunktionen:

```

seq(from = 10, to = 20, by = 2) ## Allgemeine Sequenzen

## [1] 10 12 14 16 18 20

rep(10, 3) ## Wiederholungen

```

```
## [1] 10 10 10

rep(1 : 3, 2)

## [1] 1 2 3 1 2 3

rev(1 : 3) ## Umkehrung

## [1] 3 2 1

length(1 : 5) ## Anzahl Elemente

## [1] 5

sum(1 : 5) ## Summe

## [1] 15

prod(c(2, 5, 9)) ## Produkt

## [1] 90
```

1.2 Datenmanagement

Dieser Abschnitt erklärt, wie statistische Merkmale als **R**-Objekte erfasst und transformiert werden können. Ein wichtiges Konzept ist jenes des *data frames*, mit welchem einzelne Variablen zu einem Datensatz gebündelt werden können. Den Abschluss bildet dann der Import und Export von Daten.

Als begleitendes Beispiel gehen wir davon aus, dass von vier Bekannten die Merkmale *Name*, *Alter*, *Raucht?*, *Geschlecht*, *Figur*, *Größe* und *Piercinganzahl* erhoben wurden:

Name	Alter	Raucht?	Geschlecht	Figur	Größe	#Piercings
Tim	20	Ja	männlich	mollig	1,65	1
Susi	30	Nein	weiblich	dünn	1,75	k.A.
Horst	40	Ja	männlich	normal	1,85	0
Walter	50	Nein	männlich	hager	1,95	2

1.2.1 Kategoriale Merkmale

Kategoriale Variablen sind solche, die eine begrenzte Anzahl an möglichen Ausprägungen haben.

Binäre (“Logische”) Merkmale

Das Merkmal *Raucht?* hat nur zwei mögliche Ausprägungen, und zwar die logischen Werte “Ja” und “Nein”, und ist daher binär (als Spezialfall von kategorial). In **R** gibt es einen eigenen Datentyp für “logische” Variablen:

```
raucht = c(TRUE, FALSE, TRUE, FALSE)
raucht

## [1] TRUE FALSE TRUE FALSE
```

In der Regel wird der Datentyp beim Erzeugen automatisch erraten. Statt der reservierten Wörter `TRUE` und `FALSE` können auch die Abkürzungen `T` bzw. `F` verwendet werden.

Im Gegensatz zu allgemeinen kategorialen Merkmalen machen bei logischen Werten mathematische Berechnungen fallweise Sinn, WAHR und FALSCH werden dabei als 1 bzw. 0 interpretiert. Beispielsweise kann die Anzahl der Raucher einfach mit der Summenbildung ermittelt werden:

```
sum(raucht)

## [1] 2
```

und der Raucheranteil mit dem Mittelwert:

```
mean(raucht)

## [1] 0.5
```

Strings

Kategoriale Merkmale mit mehr als zwei Ausprägungen nennt man *polytom*. Das Merkmal *Name* ist ein Grenzfall, da jede Ausprägung nur einmal vorkommt. Tatsächlich sind solche Identifikatoren für statistische Auswertungen unbrauchbar, sie taugen allenfalls für Beschriftungen. In **R** werden sie einfach als Strings (genauer: “character-Vektoren”) gespeichert.

```
name = c("Tim", "Susi", "Horst", "Walter")
name

## [1] "Tim" "Susi" "Horst" "Walter"
```

Natürlich gibt es in **R** eine Reihe von String-Funktionen, hier eine Auswahl:

```

nchar(name) # Zeichenanzahl

## [1] 3 4 5 6

substring(name, 1, 3) # Teilstrings

## [1] "Tim" "Sus" "Hor" "Wal"

toupper(name) # Grossbuchstaben

## [1] "TIM"      "SUSI"     "HORST"    "WALTER"

paste("Hallo", "Welt") # Zwei Vektoren zusammenfuegen

## [1] "Hallo Welt"

paste("Hallo", name) # Zusammenfuegen, vektorisiert

## [1] "Hallo Tim"      "Hallo Susi"     "Hallo Horst"    "Hallo Walter"

paste(name, collapse = " und ") # Mehrere Elemente zu einem zusammenfuegen

## [1] "Tim und Susi und Horst und Walter"

grep("W.*r", name) # Suche mit regular expressions, liefert Indexvektor

## [1] 4

gsub("Susi", "Susanne", name) # Ersetzen von Strings

## [1] "Tim"      "Susanne"  "Horst"    "Walter"

```

(Details auf der jeweiligen Hilfeseite.)

Polytome Merkmale

“Echte” polytome Merkmale, wie z.B. *Figur* oder *Geschlecht*², werden in **R** als sogenannte *Faktoren*³ kodiert. Als Ausgangspunkt definiert man zunächst einen String, mit dem dann der eigentliche Faktor erzeugt wird:

²*Geschlecht* könnte erstens mehr als zwei Ausprägungen haben (z.B. “unklar” oder “Transgender”), und zweitens wäre es unüblich, stattdessen eine binäre Variable “Männlich?” oder “Weiblich?” zu verwenden.

³Der Name kommt daher, dass solche Variablen in statistischen Analysen gerne als Einflussfaktoren auf andere Variablen untersucht werden.

```
g = c("M", "W", "M", "W")
geschlecht = as.factor(g)
geschlecht

## [1] M W M W
## Levels: M W
```

Bei der Ausgabe entfallen dann die Anführungszeichen, dafür werden die möglichen Kategorien (“levels”) angezeigt. Ein Vorteil von Faktoren ist, dass man die Bezeichnungen der Kategorien einfach ändern kann:

```
levels(geschlecht)

## [1] "M" "W"

levels(geschlecht) = c("Maennlich", "Weiblich")
geschlecht

## [1] Maennlich Weiblich Maennlich Weiblich
## Levels: Maennlich Weiblich
```

Aufpassen muss man bei Daten, in denen nicht alle möglichen Kategorien vorkommen (z.B. wenn man zufälligerweise nur männliche Personen befragt hat). In diesem Fall muss man den Faktor mittels der Funktion `factor()` erzeugen und die Kategorien explizit via `levels=`-Argument angeben. Optional können diese dabei auch gleich unbenannt werden (mit dem Argument `labels=`):

```
geschlecht = factor(g, levels = c("M", "W"),
                    labels = c("Maennlich", "Weiblich"))
geschlecht

## [1] Maennlich Weiblich Maennlich Weiblich
## Levels: Maennlich Weiblich
```

Ordinale Merkmale

Polytome Merkmale, für die wie bei *Figur* eine Ordnung definiert werden kann, nennt man *ordinal*. **R** kennt eine eigenen Datentyp dafür: “ordered”. Bei der Erzeugung muss die richtige Reihenfolge der Kategorien angegeben werden:

```
fig = c("mollig", "duenn", "normal", "hager")
figur = ordered(fig, levels = c("hager", "duenn", "normal", "mollig"))
figur
```

```
## [1] mollig duenn normal hager
## Levels: hager < duenn < normal < mollig
```

Manchmal möchte man bei kategorialen Variablen Ausprägungen zusammenlegen, um die Analyse zu vereinfachen. In unserem Beispiel könnten beispielsweise die Kategorien “hager” und “dünn” zu “schlank” zusammengefasst werden:

```
levels(figur)

## [1] "hager" "duenn" "normal" "mollig"

levels(figur)[1:2] = "schlank"
levels(figur)

## [1] "schlank" "normal" "mollig"

figur

## [1] mollig schlank normal schlank
## Levels: schlank < normal < mollig
```

Levels mit gleichem Namen werden zu einer Kategorie verschmolzen. Die Ursprungsinformation geht bei dieser Transformation allerdings verloren.

1.2.2 Metrische Merkmale

Zahlen haben keine speziellen Erzeugungsfunktionen, das interne Datenformat (integer oder double) wird aus den Ausprägungen “erraten”.

Ganze Zahlen

Das *Alter* wird in unserem Datensatz in ganzen Jahren⁴, also mit diskreten Werten gemessen:

```
alter = c(20, 30, 40, 50)
alter = seq(from = 20, to = 50, by = 10) ## Alternative
alter

## [1] 20 30 40 50
```

⁴Streng genommen liegt damit ein ordinales Merkmal vor, da der Abstand zwischen zwei aufeinanderfolgenden Jahren zwischen 1 und 364 Tagen sein könnte, also nicht immer gleich ist. . .

Kommazahlen

Fließkommazahlen werden in **R** immer mit doppelter Genauigkeit gespeichert. **R** erkennt diese am Dezimalpunkt (Achtung: “.”, nicht “,”!):

```
groesse = c(1.65, 1.75, 1.85, 1.95)
groesse

## [1] 1.65 1.75 1.85 1.95
```

Fallweise macht es Sinn, ein metrisches Merkmal in ein kategoriales zu transformieren (also das Skalenniveau zu reduzieren). Dabei geht zwar Information verloren, es stehen aber dafür andere Analysemethoden zur Verfügung, und manchmal ist man auch weniger an Details interessiert. Beispielsweise könnte in unserem Fall eine Einteilung von Personen in die Kategorien “klein”, “mittel” und “groß” genügen. Hierfür muss man natürlich definieren, ab welcher Größe eine Person in welche Kategorie fällt. In **R** kann man dafür die Funktion `cut()` verwenden. Diese nimmt eine sogenannte *Klasseneinteilung* vor, d.h. weist die einzelnen metrischen Werte einer von mehreren vorgegebenen Kategorien zu:

```
groesse2 = cut(groesse,
               breaks = c(1.5, 1.7, 1.8, 2.1),
               labels = c("klein", "mittel", "gross"),
               ordered = TRUE)
groesse2

## [1] klein mittel gross gross
## Levels: klein < mittel < gross
```

Das Argument `breaks=` definiert die Intervallgrenzen als linksoffene Intervalle, d.h. im Beispiel: (1.5,1.7], (1.7,1.8] und (1.8,2.1]. (Als untersten und obersten Wert kann man auch `-Inf` bzw. `+Inf` verwenden.)

1.2.3 Fehlende Werte

Susi hat beim Merkmal *Piercings* keine Angabe (“k.A.”) gemacht - ein typischer Fall eines fehlenden Wertes (“Missing Value”). Werte können aus verschiedenen Gründen fehlen (unserlich, vergessen worden, InterviewpartnerIn wollte keine Antwort geben etc.). In jedem Fall müssen solche Werte beim Erfassen extra gekennzeichnet und nicht etwa durch einen anderen, etwa 0, ersetzt werden. In **R** steht in allen Datentypen das spezielle Symbol `NA` (ähnlich einem `NULL`-Wert in relationalen Datenbanksystemen) zur Verfügung:

```

piercings = c(1, NA, 0, 2)
piercings

## [1] 1 NA 0 2

```

Fehlende Werte sind zwar bei Berechnungen “lästig”, da deren Ergebnis meist undefiniert ist, es stehen aber in der Regel geeignete Methoden zur Problembeseitigung zur Verfügung:

```

sum(piercings)

## [1] NA

sum(piercings, na.rm = TRUE) ## NAs vor der Berechnung entfernen

## [1] 3

```

1.2.4 Ein ganzer Datensatz: data frames

Erzeugen

Wir haben im vorigen Abschnitt zu Demonstrationszwecken jede Variable einzeln erzeugt - in der Regel möchte man aber den gesamten Datensatz in einem Objekt gespeichert haben, ähnlich der Tabelle, von der wir ausgegangen sind. So ein “Bündel” von Variablen - letztlich eine Datentabelle - nennt man in **R** *data frame*, und ist einer der wichtigsten komplexen Datentypen in **R** (und in allen anderen Statistik-Paketen). Ein data frame ähnelt einer Relation (Tabelle) in einem modernen Datenbanksystem: In den Zeilen stehen die einzelnen Beobachtungen (“Tupel” oder “Datensatz” im DBMS), in den Spalten die beschiedenen Variablen (“Attribute” im DBMS).

Einen leeren data frame kann man mit dem Dateneditor von **R** erzeugen:

```

daten = edit(data.frame())

```

Wenn man auf die Spaltennamen klickt, kann man Typ und Bezeichnung ändern.

Alternativ erzeugt man einen data frame aus bereits bestehenden Einzelvariablen mit der Funktion `data.frame()`:

```

freunde = data.frame(name, alter, raucht, geschlecht, figur, groesse, piercings)
freunde

```

	name	alter	raucht	geschlecht	figur	groesse	piercings
## 1	Tim	20	TRUE	Maennlich	mollig	1.65	1
## 2	Susi	30	FALSE	Weiblich	schlank	1.75	NA
## 3	Horst	40	TRUE	Maennlich	normal	1.85	0
## 4	Walter	50	FALSE	Weiblich	schlank	1.95	2

Die Spaltennamen entsprechen standardmäßig den Variablennamen, es können jedoch beim Erzeugen andere Namen vergeben werden:

```
freunde2 = data.frame(Name = name, Alter = alter, Raucht = raucht,  
                      Geschlecht = geschlecht, Figur = figur,  
                      Groesse = groesse, Piercings = piercings)
```

freunde2

##	Name	Alter	Raucht	Geschlecht	Figur	Groesse	Piercings
## 1	Tim	20	TRUE	Maennlich	mollig	1.65	1
## 2	Susi	30	FALSE	Weiblich	schlank	1.75	NA
## 3	Horst	40	TRUE	Maennlich	normal	1.85	0
## 4	Walter	50	FALSE	Weiblich	schlank	1.95	2

Alternativ können die Spaltennamen nachträglich mit `colnames()` umbenannt werden:

```
colnames(freunde2)
```

```
## [1] "Name"      "Alter"      "Raucht"      "Geschlecht" "Figur"  
## [6] "Groesse"    "Piercings"
```

```
colnames(freunde2)[1] = "NAME"
```

freunde2

##	NAME	Alter	Raucht	Geschlecht	Figur	Groesse	Piercings
## 1	Tim	20	TRUE	Maennlich	mollig	1.65	1
## 2	Susi	30	FALSE	Weiblich	schlank	1.75	NA
## 3	Horst	40	TRUE	Maennlich	normal	1.85	0
## 4	Walter	50	FALSE	Weiblich	schlank	1.95	2

Um nachträglich Spalten hinzuzufügen (beispielsweise *groesse2*), verwendet man die Funktion `cbind()` ("Column bind"):

```
freunde = cbind(freunde, groesse2)
```

freunde

##	name	alter	raucht	geschlecht	figur	groesse	piercings	groesse2
## 1	Tim	20	TRUE	Maennlich	mollig	1.65	1	klein
## 2	Susi	30	FALSE	Weiblich	schlank	1.75	NA	mittel
## 3	Horst	40	TRUE	Maennlich	normal	1.85	0	gross
## 4	Walter	50	FALSE	Weiblich	schlank	1.95	2	gross

Einzelwerte kann man bequem mit dem Dateneditor ändern:

```
fix(freunde)
```

Überblick verschaffen

Es gibt mehrere Funktionen, die Überblicksinformationen über data frames liefern. Die ersten paar Zeilen kann man ausgeben lassen mit:

```
head(freunde, 3) # default: 6 Zeilen

##      name alter raucht geschlecht  figur groesse piercings groesse2
## 1    Tim    20   TRUE  Maennlich  mollig   1.65         1    klein
## 2   Susi    30  FALSE  Weiblich  schlank   1.75        NA   mittel
## 3  Horst    40   TRUE  Maennlich  normal   1.85         0    gross
```

Die Anzahl der Zeilen und Spalten (“Dimension”) der Tabelle erhält man mit:

```
nrow(freunde) # Anzahl Zeilen

## [1] 4

ncol(freunde) # Anzahl Spalten

## [1] 8

dim(freunde) # Beides

## [1] 4 8
```

Einen “technischen” Überblick erhält man mit:

```
str(freunde)

## 'data.frame': 4 obs. of 8 variables:
## $ name      : chr  "Tim" "Susi" "Horst" "Walter"
## $ alter     : num  20 30 40 50
## $ raucht    : logi  TRUE FALSE TRUE FALSE
## $ geschlecht: Factor w/ 2 levels "Maennlich","Weiblich": 1 2 1 2
## $ figur     : Ord.factor w/ 3 levels "schlank"<"normal"<...: 3 1 2 1
## $ groesse   : num  1.65 1.75 1.85 1.95
## $ piercings : num  1 NA 0 2
## $ groesse2  : Ord.factor w/ 3 levels "klein"<"mittel"<...: 1 2 3 3
```

Man sieht für jede Variable den Datentyp, sowie die ersten paar Werte.

Eine Zusammenstellung von einfachen Kennzahlen bekommt man mit:

```
summary(freunde)
```

```
##      name      alter      raucht      geschlecht      figur
## Length:4      Min.    :20.0      Mode :logical      Maennlich:2      schlank:2
## Class :character 1st Qu.:27.5      FALSE:2      Weiblich :2      normal :1
## Mode :character  Median :35.0      TRUE :2      mollig :1
##                      Mean   :35.0
##                      3rd Qu.:42.5
##                      Max.    :50.0
##
##      groesse      piercings      groesse2
## Min.    :1.650      Min.    :0.0      klein :1
## 1st Qu.:1.725      1st Qu.:0.5      mittel:1
## Median :1.800      Median :1.0      gross :2
## Mean   :1.800      Mean   :1.0
## 3rd Qu.:1.875      3rd Qu.:1.5
## Max.    :1.950      Max.    :2.0
##                      NA's    :1
```

Für kategoriale Variablen erhält man die Anzahl der Beobachtungen in den jeweiligen Kategorien, für metrische Variablen die Fünf-Punkt-Zusammenfassung (Mittelwert/Median, Minimum, Maximum, sowie 1. und 3. Quartil) - Details hierzu folgen in den weiteren Kapiteln. Des weiteren sieht man die Anzahl fehlender Werte ("NA's") in den jeweiligen Variablen, falls vorhanden.

Indexieren

Auf einzelne Zeilen, Spalten und Elemente eines data frame kann man mit Hilfe des Index-Operators (`[]`) zugreifen:

```
freunde[1,] # erste Zeile (ein data frame)

##      name alter raucht geschlecht  figur groesse piercings groesse2
## 1   Tim    20   TRUE  Maennlich mollig    1.65         1     klein

freunde[,1] # erste Spalte (ein character-Vektor)

## [1] "Tim"      "Susi"      "Horst"     "Walter"

freunde[2,2] # Element an der zweiten Zeile und Spalte

## [1] 30
```

```
freunde[2:3,] # Auswahl von Zeilen 2 und 3
```

	name	alter	raucht	geschlecht	figur	groesse	piercings	groesse2
## 2	Susi	30	FALSE	Weiblich	schlank	1.75	NA	mittel
## 3	Horst	40	TRUE	Maennlich	normal	1.85	0	gross

Auf einzelne Spalten (Variablen) kann man auch mit dem Variablennamen zugreifen:

```
freunde[, "name"]
```

	name
## [1]	"Tim" "Susi" "Horst" "Walter"

```
freunde$name # kuerzeste Form
```

	name
## [1]	"Tim" "Susi" "Horst" "Walter"

Durch Indexieren kann man auch Spalten (oder Zeilen) umordnen:

```
freunde = freunde[, c(1,2,3,4,5,6,8,7)]
freunde
```

	name	alter	raucht	geschlecht	figur	groesse	groesse2	piercings
## 1	Tim	20	TRUE	Maennlich	mollig	1.65	klein	1
## 2	Susi	30	FALSE	Weiblich	schlank	1.75	mittel	NA
## 3	Horst	40	TRUE	Maennlich	normal	1.85	gross	0
## 4	Walter	50	FALSE	Weiblich	schlank	1.95	gross	2

groesse2 steht nun neben *groesse*.

Filtern

Wie bei Datenbanken (z.B. mit dem SQL-select-Statement) kann man mit logischen Ausdrücken filtern. Hierfür steht die Funktion `subset()` zur Verfügung, mit der man sowohl Zeilen (Selektion) wie auch Spalten (Projektion) filtern kann.

```
subset(freunde, alter > 25 & raucht == TRUE)
```

	name	alter	raucht	geschlecht	figur	groesse	groesse2	piercings
## 3	Horst	40	TRUE	Maennlich	normal	1.85	gross	0

```
subset(freunde, raucht, c(name, alter)) ## == TRUE kann weggelassen werden
```

	name	alter
## 1	Tim	20
## 3	Horst	40

```
subset(freunde, , alter)
```

```
##      alter
## 1      20
## 2      30
## 3      40
## 4      50
```

Achtung, für das logische “Und” wird nur ein einfaches & benutzt.

Um Beobachtungen mit fehlenden Werten zu entfernen, kann man - nach vorheriger Prüfung, dass nur wenige Beobachtungen verloren gehen! - die Prüffunktion `complete.cases()` verwenden:

```
ok = complete.cases(freunde)
sum(ok) # Anzahl vollstaendiger Beobachtungen

## [1] 3

sum(!ok) # Anzahl unvollstaendiger Beobachtungen

## [1] 1

freunde_complete = subset(freunde, ok) # alle vollstaendigen extrahieren
freunde_complete

##      name alter raucht geschlecht  figur groesse groesse2 piercings
## 1    Tim    20   TRUE  Maennlich  mollig    1.65    klein      1
## 3  Horst    40   TRUE  Maennlich  normal    1.85    gross      0
## 4 Walter    50  FALSE   Weiblich  schlank    1.95    gross      2

subset(freunde, !ok)

##      name alter raucht geschlecht  figur groesse groesse2 piercings
## 2  Susi    30  FALSE   Weiblich  schlank    1.75    mittel     NA
```

Aggregieren

Ähnlich wie mit SQL⁵ ist es möglich, Zeilen zu gruppieren und metrische Variablen mit Aggregatsfunktionen zusammenzufassen:

⁵`select sum(FOO), BAR from BUZ group by BAR;`

```
## Die meisten Piercings nach Geschlecht
aggregate(piercings ~ geschlecht, data = freunde, FUN = max)

##      geschlecht piercings
## 1  Maennlich          1
## 2   Weiblich          2

## durchschnittliche Groesse nach Figur
aggregate(groesse ~ figur, freunde, mean)

##      figur groesse
## 1 schlank    1.85
## 2  normal    1.85
## 3  mollig    1.65

## durchschnittliche Groesse nach Figur und Geschlecht
aggregate(groesse ~ figur + geschlecht, freunde, mean)

##      figur geschlecht groesse
## 1  normal  Maennlich    1.85
## 2  mollig  Maennlich    1.65
## 3 schlank   Weiblich    1.85

## Durchschnitt von Groesse und Alter nach Geschlecht
aggregate(cbind(groesse, alter) ~ geschlecht, freunde, mean)

##      geschlecht groesse alter
## 1  Maennlich    1.75     30
## 2   Weiblich    1.85     40
```

Das erste Argument erwartet eine sogenannte *Modellformel* - die Tilde (“~”) bedeutet: “auswerten nach”. Links der Tilde stehen jene Variablen, die aggregiert werden sollen (die Funktion `cbind()`, normalerweise zum verbinden von zwei Variablen zu einer Tabelle, heißt hier sinngemäß: “gemeinsam auswerten”). Auf der rechten Seite der Tilde stehen die Gruppierungsvariablen (Das Plus-Zeichen (“+”) heißt nicht addieren, sondern einfach nur: “und”).

1.2.5 Datenimport und -export

In der Praxis wird man Daten selten in **R** händisch erfassen, sondern aus einer anderen Quelle (z.B. aus einem webbasierten Umfragetool) importieren. Die einfachsten und gängigsten Formate sind Textdateien. Im folgenden unterscheiden wir zwischen Daten für Einzelvariablen (also ein einziges Merkmal soll eingelesen oder geschrieben werden), und Daten für mehrere

Merkmale (also Daten, die typischerweise als data frame vorliegen, wie im Freunde-Beispiel).

Exportieren eines Einzelmerkmals

Eine einzelne Variable kann mit der Funktion `write()` in eine Datei geschrieben werden:

```
x = (1 : 20) / 10
write(x, file = "x.txt", sep = ",")
```

Dies erzeugt folgende Datei:

```
0.1,0.2,0.3,0.4,0.5
0.6,0.7,0.8,0.9,1
1.1,1.2,1.3,1.4,1.5
1.6,1.7,1.8,1.9,2
```

Bei Zahlen werden pro Zeile standardmäßig 5 Werte pro Zeile geschrieben⁶, getrennt durch das `sep`-Zeichen. Bei String-Vektoren wird standardmäßig immer ein String pro Zeile geschrieben, da dieser ja Leerzeichen oder Beistriche enthalten könnte - die Trennung der einzelnen Elemente erfolgt also durch den Zeilenumbruch.

Vorsicht ist beim Export von Faktoren geboten: Intern werden diese als Zahlen gespeichert und mittels `write()` auch als solche geschrieben. Da damit die Bezeichnung der Kategorien verloren geht, empfiehlt sich vor dem Export die Umwandlung in Strings:

```
write(as.character(geschlecht), file = "geschlecht.txt")
```

Importieren eines Einzelmerkmals

Das Gegenstück zu `write()` ist die Funktion `scan()`. Diese kann von beliebigen "Streams" lesen, also nicht nur von lokal gespeicherten Dateien, sondern auch aus dem Internet (durch Angabe eines URLs) oder von der Tastatur. Im Gegensatz zu `write()` muss man einige Optionen mehr setzen, da die unterschiedlichen Varianten (Trennzeichen, Zeichensatz-Kodierung, Variablentyp ...) nicht automatisch erkannt werden können. Es empfiehlt sich daher, Textdateien vor dem Importieren kurz anzusehen, um die richtigen Importeinstellungen zu wählen. Zunächst muss man dafür den Pfad zur Textdatei ermitteln.

Variante 1: man ermittelt zunächst das aktuelle Arbeitsverzeichnis von **R**:

```
getwd()

## [1] "/home/marcus/Dropbox/DSM/DE/Datenmanagement_in_R"
```

⁶Dies kann mit dem Argument `ncolumns=` geändert werden.

und kopiert die Datei dorthin - im Beispiel: `x.txt`. Kontrolle in **R** mit:

```
dir(pattern = "*.txt")  
  
## [1] "geschlecht.txt" "raucht.txt"      "x.txt"
```

Variante 2: man sucht aus **R** heraus die Datei mit einem File-Browser, und speichert den Pfad in einer Variable:

```
path = file.choose()
```

Um sich die Datei anzusehen, verwendet man am besten:

```
file.show(path)
```

Achtung! Übliche Editoren unter Windows wie "Notepad", oder gar Excel, sind für die Betrachtung von Textdateien *ungeeignet*, da sie Sonderzeichen (z.B. Zeilenumbrüche und Umlaute) unter Umständen falsch darstellen, speziell wenn die Dateien von anderen Betriebssystemen (Linux, Mac) stammen.

Nun kann die Datei mit Hilfe der Funktion `scan()` und entsprechenden Argumenten eingelesen werden:

```
my_x = scan(path, sep = ",", dec = ".")  
my_x  
  
## [1] 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0 1.1 1.2 1.3 1.4 1.5 1.6 1.7 1.8 1.9  
## [20] 2.0
```

Standardmäßig nimmt `scan()` an, dass man Zahlen einlesen möchte. Im Falle von Strings muss dies explizit angegeben werden, sonst erhält man eine Fehlermeldung:

```
scan("geschlecht.txt")  
  
## Error in scan("geschlecht.txt"): scan() erwartete 'a real', bekam  
'Maennlich'  
  
scan("geschlecht.txt", what = "A")  
  
## [1] "Maennlich" "Weiblich" "Maennlich" "Weiblich"
```

(Beim Argument `what=` muss einfach irgendein String angegeben werden. Für Zahlen (default) könnte man irgendeine Zahl verwenden, z.B. 1). Speziell beim Einlesen von Strings können noch weitere Stolpersteine auftreten: sollten Umlaute nicht korrekt dargestellt werden, stimmt oft das Datei-Encoding nicht mit jenem des Betriebssystems überein - Abhilfe schafft meist die

Option `fileEncoding="UTF-8"` oder `fileEncoding="Latin-1"`. Manchmal sind Strings auch durch unterschiedlich viele Leerzeichen getrennt, welche dann teilweise als vermeintliche Daten miteingelesen werden: in diesem Fall kann man das Argument `strip.white = TRUE` verwenden. Weitere Optionen entnehme man der Hilfeseite für `scan()`.

Schließlich sei noch erwähnt, dass man mit `scan()` auch von der Tastatur einlesen kann. Dies kann für die rasche Erfassung kleinerer Datenmengen genutzt werden:

```
x = scan()
1: 44
2: 22 33
4:
Read 3 items
```

(Letzte Zeile mit ENTER bestätigen). Dies liest die drei Werte 44, 22 und 33 ein. Wie man sieht, kann man auch mehrere Werte in eine Zeile schreiben, sofern diese durch "Whitespace" (Leerzeichen) getrennt sind. Damit kann man beispielsweise Zahlen aus einem elektronischen Dokument einfach mittels copy/paste nach **R** bringen.

Exportieren von data frames

Daten mit mehreren Merkmalen - also data frames - werden typischerweise in Textdateien gespeichert, in denen jede Zeile eine Beobachtung darstellt, und die einzelnen Merkmalsausprägungen mit einem Trennzeichen separiert sind ("CSV-Dateien" für "Comma-Separated-Values"), z.B.:

```
"name","alter","raucht","geschlecht","figur","groesse","groesse2","piercings"
"Tim",20,TRUE,"Maennlich","mollig",1.65,"klein",1
"Susi",30,FALSE,"Weiblich","schlank",1.75,"mittel",NA
"Horst",40,TRUE,"Maennlich","normal",1.85,"gross",0
"Walter",50,FALSE,"Weiblich","schlank",1.95,"gross",2
```

Hierfür wird in **R** die Funktion `write.table()` verwendet:

```
write.table(freunde, file = "freunde.csv", sep = ",", row.names = FALSE)
```

Das Argument `sep=` gibt an, dass die einzelnen Merkmalsausprägungen mit Komma zu trennen sind. `row.names=FALSE` verhindert, dass auch die Zeilen-IDs mitgespeichert werden - da hierfür keine eigene Spaltenüberschrift erzeugt wird, macht das beim Einlesen auf anderen Systemen als **R** (z.B. in Excel) Probleme. Andere nützliche Argumente (mit ihren Defaultwerten) sind: `dec="."` (Dezimalpunkt), `eol="\n"` (Zeilenumbruch) und `col.names = TRUE` (Spaltenüberschriften (J/N)?).

Importieren von data frames

Die "Umkehrfunktion" zu `write.table()` ist `read.table()`. Es gilt sinngemäß das für `scan()` angegebene: Zunächst sollte man sich die Datei mit `file.show()` ansehen, um dann die Argumente richtig setzen zu können. Diese sind ähnlich zu `scan()`:

```
my_freunde = read.table("freunde.csv", header = TRUE, sep = ",", dec = ".")
my_freunde
```

##	name	alter	raucht	geschlecht	figur	groesse	groesse2	piercings
## 1	Tim	20	TRUE	Maennlich	mollig	1.65	klein	1
## 2	Susi	30	FALSE	Weiblich	schlank	1.75	mittel	NA
## 3	Horst	40	TRUE	Maennlich	normal	1.85	gross	0
## 4	Walter	50	FALSE	Weiblich	schlank	1.95	gross	2

Nützliche zusätzliche Argumente sind beispielsweise `fileEncoding=` (siehe `scan()`) und `na.strings=` zur Angabe der Labels für fehlende Werte in der Quelldatei (z.B. "k.A. ").

Standardmäßig wurden Strings bei Verwendung von `read.table` in der Vergangenheit in Faktoren umgewandelt, aber ab **R** 4.0.0 wurde die entsprechende Standardeinstellung vor allem aus Gründen der Reproduzierbarkeit geändert⁷. Will man dieses Verhalten wiederherstellen, dann lässt sich das durch Setzen von `stringsAsFactors = TRUE` erreichen. Um es für bestimmte Spalten trotzdem zu verhindern (beispielsweise für *Name*), kann man mit `as.is=` jene Spalte(n) (der Quelldatei!) angeben, für die einfach nur eine String-Variable entstehen soll:

```
my_freunde = read.table("freunde.csv", stringsAsFactors = TRUE,
                        as.is = 1, header = TRUE, sep = ",")
```

```
str(my_freunde)
```

```
## 'data.frame': 4 obs. of 8 variables:
## $ name      : chr  "Tim" "Susi" "Horst" "Walter"
## $ alter     : int   20  30  40  50
## $ raucht    : logi  TRUE FALSE TRUE FALSE
## $ geschlecht: Factor w/ 2 levels "Maennlich","Weiblich": 1 2 1 2
## $ figur     : Factor w/ 3 levels "mollig","normal",...: 1 3 2 3
## $ groesse   : num   1.65 1.75 1.85 1.95
## $ groesse2  : Factor w/ 3 levels "gross","klein",...: 2 3 1 1
## $ piercings : int    1 NA  0  2
```

⁷Dasselbe gilt für `data.frame()`: String-Vektoren wurden beim Erstellen eines Data Frames in **R**-Versionen < 4.0.0 automatisch in Faktoren konvertiert.

1.3 Weiterführende Themen

1.3.1 Funktionen

In Abschnitt 1.1.3 haben wir gesehen, wie einfache (mathematische) Funktionen definiert werden:

```
quadrat = function(x) x * x
```

Eine Funktion hat als Rückgabewert den Wert des letzten Ausdrucks. In diesem Falle gibt es nur einen $(x * x)$, somit wird das Ergebnis der Rechenoperation retourniert.

R-Funktionen können natürlich komplexer sein, und insbesondere mehrere Argumente besitzen. Diese können Default-Werte besitzen, und beim Aufruf entweder positional oder mit Namen gesetzt werden. Die folgende Funktion begrüßt den Aufrufer entweder auf deutsch oder englisch. Die Grußformel ist standardmäßig "Hallo" bzw. "Hello", kann aber bei Bedarf verändert werden:

```
greet <- function(name,
                  language = "DE",
                  text = c(DE = "Hallo", EN = "Hello"))
{
  cat(text[language], ", ", name, "!\n", sep = "")
}
greet("David")

## Hallo, David!

greet("David", "EN")

## Hello, David!

greet("David", text = c(DE = "Morgen", EN = "Morning"))

## Morgen, David!
```

R-Funktionen können mit der Funktion `fix()` einfach editiert werden:

```
fix(greet)
```

Natürlich können in **R** Bedingungen formuliert werden:

```
greet <- function(name,
                  language = "DE",
```

```

        text = c(DE = "Hallo", EN = "Hello"))
{
  if (language != "DE" && language != "EN")
    cat("Wrong argument!\n")
  else
    cat(text[language], ", ", name, "!\n", sep = "")
}
greet("David", lang = "ES")

## Wrong argument!

```

Schleifen sind zwar in **R** selten nötig, aber möglich. Der Ausdruck `for(i in Vec)` *Anweisung* iteriert über alle Werte in `Vec`:

```

greet <- function(name,
  language = "DE",
  text = c(DE = "Hallo", EN = "Hello"),
  times = 1)
{
  if (language != "DE" && language != "EN")
    cat("Wrong argument!\n")
  else {
    for (i in 1 : times)
      cat(text[language], ", ", name, "!\n", sep = "")
  }
}
greet("David", times = 3)

## Hallo, David!
## Hallo, David!
## Hallo, David!

```

1.3.2 Grafik

R ist ein sehr mächtiges Grafiksystem, mit dem professionelle Vektorgrafiken generiert werden können. Das folgende Beispiel gibt eine Idee dazu:

```

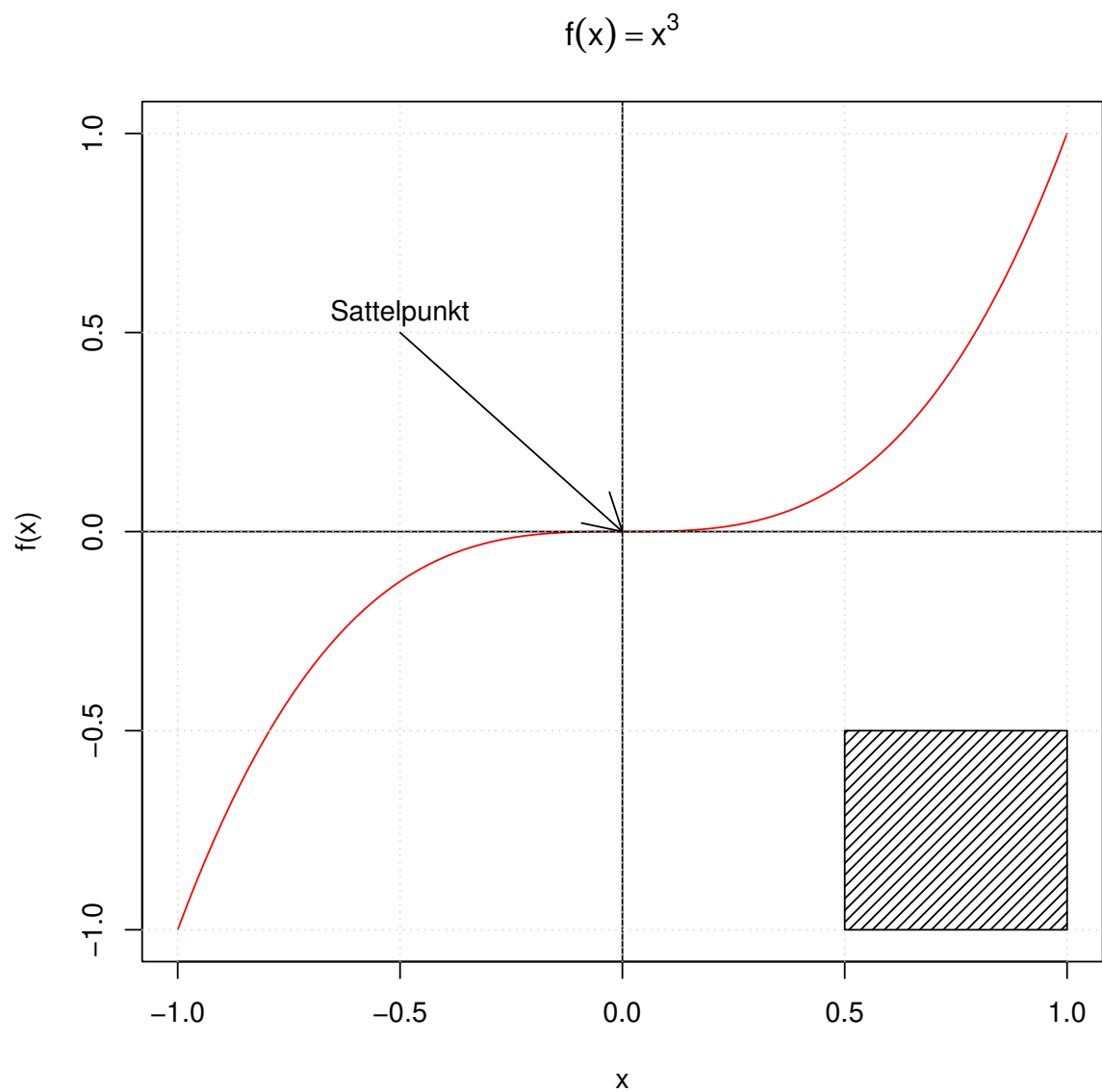
f = function(x) x ^ 3
curve(f, from = -1, to = 1, col = "red",
  main = expression(f(x) == x^3))

```

```
## Etwas Kosmetik:
abline(h = 0, v = 0) ## X- und Y-Achse
grid() ## Raster

## Spezialbeschriftungen
arrows(x0 = -0.5, y0 = 0.5, x1 = 0, y1 = 0)
text(x = -0.5, y = 0.55, labels = "Sattelpunkt")

## Weitere Grafikbefehle: zB lines() oder rect():
rect(xleft = 0.5, ybottom = -1, xright = 1, ytop = -0.5, density = 20)
```



Weitere Beispiele erhält man mit

1.4 Die wichtigsten R-Funktionen im Überblick

1.4.1 Allgemeines

? Hilfe (z.B. `?cat`)

=, <- Zuweisung (z.B. `x = 2`)

cat() Ausgabe von Werten (z.B. `cat("Hallo", "Welt")`)

1.4.2 Erzeugen von Variablen

c() Erzeugen von Vektoren (z.B. `x = c(1, 2, 3)`, `x = c(a = 1, b = 2)`)

a : b Sequenzoperator (z.B. `1 : 5`)

seq() Allgemeine Sequenzen (z.B. `x = seq(from = 10, to = 100, by = 10)`)

rep() Wiederholen von Vektoren (z.B. `x = rep(c(1, 2), 4)`)

rev() Umkehren von Vektoren (z.B. `x = rev(1:3)`)

length() Länge eines Vektors (z.B. `length(1:3)`)

factor(), as.factor() Erzeugen eines Faktors aus einem String (z.B. `x = factor(c("A", "B"))`)

ordered() Erzeugen eines ordinalen Merkmals aus einem String (z.B. `x = ordered(c("b", "a"), labels = c("a", "b"))`)

levels() Extrahieren/Verändern der Kategorien eines Faktors (z.B. `levels(x)`, `levels(x) = c("a", "b")`)

cut() Umwandeln eines metrischen in ein kategoriales Merkmal (z.B. `cut(1:10, breaks = c(-Inf, 5, +Inf), levels = c("bis_5", "groesser_5"))`)

1.4.3 String-Funktionen

nchar() Zeichenanzahl pro Element (z.B. `nchar(c("ab", "c"))`)

substring() Teilstrings extrahieren (z.B. `substring(c("ABC", "DEFG"), 2, 3)`)

paste() String-Elemente zusammenfügen (z.B. `x = paste("Hallo", "Welt")`)

toupper(), tolower() Strings in Großbuchstaben/Kleinbuchstaben umwandeln (z.B. `tolower("Hallo")`)

grep() Suchen mit regular expressions (z.B. `grep("H.*o", c("Hallo", "Welt"))`)

gsub() Ersetzen von Werten (z.B. `gsub("Hallo", "Servus", c("Hallo", "Welt"))`)

1.4.4 Data Frames

data.frame() Erzeugen eines data frame aus Einzelvariablen (z.B. `x = data.frame(Var1, Var2)`)

edit() Erstellen eines neuen Objekts (z.B. `x = edit(data.frame())`)

fix() Editieren eines Objekts (z.B. `fix(x)`)

cbind() Spalten zu einem data frame hinzufügen (z.B. `x = cbind(x, Var3)`)

nrow(), ncol(), dim() Anzahl Zeilen/Spalten eines data frames ermitteln (z.B. `nrow(x)`)

colnames() Spaltenüberschriften anzeigen bzw. ändern (z.B. `colnames(x) = c("a", "b")`)

head() Die ersten paar Zeilen eines data frame ausgeben (z.B. `head(x), head(x, 2)`)

str() Variablentypen eines data frame ausgeben

summary() Für jedes Merkmal in einem data frame Kennzahlen ermitteln (z.B. `summary(x)`)

[] Zeilen/Spalten/Elemente eines data frame extrahieren (z.B. `x[2, ], x[, 1:2], x[2, 3]`)

subset() Filtern von Zeilen mit logischen Ausdrücken (z.B. `subset(x, Var1 == "a" & Var2 > 2)`)

complete.cases() Ermitteln der vollständigen Beobachtungen in einem data frame (z.B. `complete.cases(x)`)

aggregate() Gruppieren von metrischen Werten nach Faktoren (z.B. `aggregate(zahl ~ faktor1 + faktor2, data = x, FUN = mean)`)

1.4.5 Workspace-Management

ls() Anzeigen aller Elemente (z.B. `ls()`)

rm() Löschen eines oder mehrerer Objekte (z.B. `rm(a, b), rm(list = ls())`)

save() Speichern eines oder mehrerer Objekte (z.B. `save(a, file = "a.rda")`)

save.image() Speichern des gesamten Workspace (z.B. `save.image()`)

load() Laden von gespeicherten Objekten (z.B. `load("a.rda")`)

1.4.6 Import / Export

file.choose() Öffnen eines File-Browsers zur Ermittlung des Pfades einer Datei (z.B. `path = file.choose()`)

file.show() Anzeigen einer Textdatei (z.B. `file.show(path)`)

write() Speichern eines Einzelmerkmals (z.B. `write(x, file = "x.txt", sep=",")`)

scan() Einlesen eines Einzelmerkmals aus einer Datei oder der Tastatur (z.B. `x = scan(file = "x.txt", sep = ",")`)

write.table() Speichern eines data frames in einer Datei (z.B. `write.table(x, header = TRUE, file = "x.csv")`)

read.table() Einlesen eines data frames von einer Datei (z.B. `x = read.table(file = "x.csv", sep = ",")`)

1.5 Übungsbeispiele

1.5.1 Einfache Vektoroperationen

- a) Erzeugen Sie einen Vektor `ALTER` mit folgenden Werten: 10, 12, 44, 78, 20, 1.
- b) Erzeugen Sie daraus einen Vektor `GEBURTSJAHR` und geben Sie ihn aus!
- c) Rechnen Sie das durchschnittliche Alter aus!
- d) Extrahieren Sie aus `ALTER` jedes zweite Element und geben Sie das Ergebnis aus! (Hinweis: erzeugen Sie erst einen Indexvektor als Sequenz von 1 bis zur halben Länge von `ALTER`, und multiplizieren Sie diesen dann mit 2).
- e) Entfernen Sie jedes dritte Element aus dem Vektor permanent!

1.5.2 Erzeugen von data frames

Gegeben sei folgende Tabelle:

(k.A. = keine Angabe)

1. Erstellen Sie die Variablen
name als character string

	Name	Figur	Groesse	Gewicht	Trinkt	Beruf
1	Susi	mollig	1.97	98	TRUE	Student
2	Tim	mollig	1.67	89	k.A.	Student
3	Christine	muskulös	1.90	71	TRUE	Student
4	Mathias	hager	1.81	86	TRUE	angestellt

beruf als Faktor mit den Ausprägungen: *Student, angestellt, selbständig*

figur als geordneter Faktor mit den Ausprägungen: *hager, dünn, schlank, normal, mollig, muskulös*

groesse als metrische (kontinuierliche) Variable

gewicht als metrische (diskrete) Variable

trinkt als logische (binäre) Variable

Achten Sie auf die korrekte Spezifikation fehlender Werte!

- Erstellen Sie aus diesen Variablen einen data frame `Freunde`, welcher der Angabe möglichst entspricht, und geben Sie ihn aus!

1.5.3 Filtern und Aggregieren von data frames

In **R** steht Ihnen der Datensatz `chickwts` zur Verfügung, welcher das Gewicht von Hühnern in Abhängigkeit von verschiedenen Futtermitteln enthält. Es handelt sich um einen data frame mit 2 Variablen (`weight` und `feed`).

- Zeigen Sie die ersten 6 Zeilen des Datensatzes an.
- Extrahieren Sie alle Faktorstufen der Variable `feed`.
- Extrahieren Sie alle Zeilen des data frame für Hühner, die mit **meatmeal** gefüttert wurden (Hinweis: `subset()`).
- Extrahieren Sie das Gewicht jener Hühner, die mit **casein** oder **horsebean** gefüttert wurden (Hinweis: Operator für logisches "oder": `|`).
- Rechnen Sie das durchschnittliche Gewicht für jedes Futtermittel aus (Hinweis: `aggregate()`).

1.5.4 Variablentransformationen

Verwenden Sie wiederum den Datensatz `chickwts`.

- Rechnen Sie das mittlere Gewicht aller Hühner aus (Hinweis: Funktion `mean()`).

2. Erzeugen Sie aus `weight` eine kategoriale Variable `gewicht` mit drei ungefähr gleich großen Kategorien `leicht`, `mittel`, `schwer` (Hinweis: `cut()`).
3. Legen Sie anschließend die Kategorien `leicht` und `mittel` zu einer neuen Kategorie `Standard` zusammen, und benennen Sie die Kategorie `schwer` auf `Premium` um.
4. Wie viele "Premium"-Hühner gibt es, die mit **meatmeal** gefüttert wurden? (Nicht händisch abzählen, sondern mit **R** ermitteln!)

1.5.5 Datenimport und -export

1. Lesen Sie folgende Werte in die Variable `x` mit Hilfe der Funktion `scan` ein:
12 3 3 34 43 23 34 90
2. Lesen Sie folgende Wörter in die Variable `y` mit Hilfe der Funktion `scan` ein ("," ist Trennzeichen):
A;quick;brown;fox;jumps;over;the;lazy;dog
3. Speichern Sie die Tabelle `Freunde` aus Beispiel 1.5.2 als komma-separierte Datei `daten.csv` ab (Trennzeichen: ";"), wobei weder Zeilen- noch Spaltenbeschriftungen abgespeichert werden sollen!
4. Lesen Sie die Textdatei `daten.csv` in die Tabelle `tab` mit Hilfe der Funktion `read.table` ein und zeigen Sie die ersten paar Zeilen an! (Hinweis: Zeigen Sie sie Datei zuvor in einem Editor oder mit `file.show()` an!)

1.5.6 Import von data frames

Führen Sie folgendes Skript in **R** via copy/paste aus:

```
library(MASS)
params = expand.grid(sep = c(" ", "|", ";"),
                    na = c("NA", "k.A."),
                    dec = c(".", ","),
                    row.names = c(TRUE, FALSE),
                    col.names = c(TRUE, FALSE),
                    fileEncoding = c("UTF-8", "latin1"),
                    stringsAsFactors = FALSE)
for (i in sample(1 : nrow(params), 10))
  do.call(write.table, c(list(x = Cars93,
                             file = paste("data", i, ".txt", sep = ""),
                             quote = TRUE),
                        params[i,]))
```

Es erzeugt 10 Textdateien, basierend auf dem Datensatz *Cars93*, welcher in **R** verfügbar ist (siehe Hilfeseite mit `?Cars93`). Lesen Sie alle 10 Dateien in **R** ein!